Computación Paralela y Distribuída

2026-II

José Fiestas PhD

16 de septiembre de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 4: Comunicación y coordinación

Objetivos:

1. Pasos de Mensaje: MPI, Mensajes Punto a Punto, MPI, Comunicación Colectiva,
2. Blocking vs non-blocking
3. Tipos MPI
4. Memoria Compartida: OMP, Constructores y cláusulas
5. Programacion Hibrida

Temática

OpenMP (Open Multi-processing)

Constructor Master

Especifica un bloque que es ejecutado por el *master thread*, dentro de la región en paralelo correspondiente. En el ejemplo, el master controla el número de iteraciones e imprime un reporte de avance de la ejecución. Los demas *threads* evitan la región master sin necesidad de esperar.

```
1  #pragma omp master new-line
2      structured-block
```

Ejemplo 01: Constructor Master

```
1  void master_example(float* x, float* xold, int n, float tol){
2  int c=0, i, toobig;
3  float error, y;
4  #pragma omp parallel
5  {       do{
6      #pragma omp for private(i)
7          for( i = 1; i < n-1; ++i ){
8          xold[i] = x[i];
9      }
10     #pragma omp single
11     {    toobig = 0;
12     }
13     #pragma omp for private(i,y,error) reduction(+:toobig)
14         for( i = 1; i < n-1; ++i ){
15             y = x[i];
16             x[i] = average( xold[i-1], x[i], xold[i+1] );
17             error = y - x[i];
18             printf("i: %d , hilo: %d\n ", i,omp_get_thread_num());
19             if( error > tol || error < -tol ) ++toobig;
20         }
21     #pragma omp master
22     {    ++c;
23         printf( "iteration %d, toobig=%d\n", c, toobig);
24     }
25     }while( toobig $>$ 0 );
26 } }
```

Constructor Critical

Restringe la ejecución del bloque a un *thread* a la vez. Influencia a todos los *threads* dentro de la región en paralelo. Se le puede asignar un nombre. Un *thread* esperará el inicio de una región crítica, hasta que no haya *threads* ejecutando esa región. En el ejemplo, una tarea es sacada de la cola (dequeue) y trabajada. La tarea debe ser crítica para evitar que la misma tarea se ejecute por diferentes *threads*. Aqui, dos colas independientes se nombran *xaxis e yaxis*.

```
int dequeue(float *a);
void work(int i, float *a);
void critical_example(float *x, float *y)
{
int ix_next, iy_next;
#pragma omp parallel shared(x, y)
private(ix_next, iy_next)
{
#pragma omp critical (xaxis)
ix_next = dequeue(x);
work(ix_next, x);
#pragma omp critical (yaxis)
iy_next = dequeue(y);
work(iy_next, y);
}}
```

Regiones Barrier dependientes

Un constructor **barrier** enlazará la región en paralelo mas inmediata. Todos los *threads* deben ejecutar **barrier** y las tareas previas antes de poder continuar. En el ejemplo, el barrier en *sub3* esta ligada a la región paralela en *sub2*.

```
void work(int n) {}
void sub3(int n) {
work(n);
#pragma omp barrier
work(n);
}
void sub2(int k) {
#pragma omp parallel shared(k)
sub3(k);
}
void sub1(int n) {
int i;
#pragma omp parallel private(i) shared(n)
{
#pragma omp for
for (i=0; i<n; i++)
sub2(i);
}
}
```

Constructor atomic

Asegura que la locación de memoria será accedida atomicamente, evitando lectura o escritura simultánea de la misma, lo que podría conducir a valores erróneos. **Atomic** fuerza acceso exclusivo con respecto a otras regiones **atomic** que acceden a la misma locación de memoria x, entre todos los *threads* del programa, independientemente de los bloques a los que pertenecen.

#pragma omp atomic capture *new-line*
structured-block

Constructor atomic

#pragma omp atomic [read | write | update | capture] *new-line*
expression-stmt

La cláusula **read** fuerza una lectura atómica de la ubicación de almacenamiento de **x**.

```
1   v=x; // v es escalar que no accede al espacio en memoria de x
```

La cláusula **write** fuerza una escritura atómica en la ubicación de almacenamiento de **x**.

```
1   x = expr; // expr es expresion de tipo escalar
```

Constructor atomic

#pragma omp atomic [read | write | update | capture] *new-line*
expression-stmt

La cláusula **update** fuerza una actualización atómica en la ubicación de almacenamiento de **x**.

```
1  x++;
2  x--;
3  ++x;
4  --x;
5  x binop= expr;  // binop es uno de +, *, -, \/, &, ^, |, <<, >>
6  x = x binop expr;
7  x = expr binop x;
```

Constructor atomic

#pragma omp atomic [read | write | update | capture] *new-line*
expression-stmt

La cláusula **capture** fuerza una actualización atómica en la ubicación de almacenamiento de **x**, a la vez que la captura del valor original o final de **x**, con respecto al **atomic** update.

```
1  v = x++;
2  v = x--;
3  v = ++x;
4  v = --x;
5  v = x binop= expr;
6  v = x = x binop expr;
7  v = x = expr binop x;
```

Constructor atomic

#pragma omp atomic [read | write | update | capture] *new-line*
expression-stmt

Se pueden agregar bloques de código, con las siguientes características

```
1   { v = x; x binop= expr; }
2   { x binop= expr; v = x; }
3   { v = x; x = x binop expr; }
4   { v = x; x = expr binop x; }
5   { x = x binop expr; v = x; }
6   { x = expr binop x; v = x; }
7   { v = x; x = expr; }
8   { v = x; x++; }
9   { v = x; ++x; }
10  { ++x; v = x; }
11  { x++; v = x; }
12  { v = x; x--; }
13  { v = x; --x; }
14  { --x; v = x; }
15  { x--; v = x; }
```

Mas detalle en especificación OpenMP

Constructor atomic

En el ejemplo,
se evitan condiciones de competencia (actualización simultánea de **x** por múltiples *threads*) usando el constructor **atomic**.
Si se usara **critical**,
los updates se ejecutarían secuencialmente

```
float work1(int i) {
return 1.0 * i;
}
float work2(int i) {
return 2.0 * i;
}
void atomic_example(float *x, float *y, int
*index, int n) {
int i;
#pragma omp parallel for shared(x, y, index,
n)
for (i=0; i<n; i++) {
#pragma omp atomic update
x[index[i]] += work1(i);
y[i] += work2(i);
}
}
```

Constructor atomic

Note que
las directivas **atomic** se aplican solo a los bloques que le siguen.
Como consecuencia, los elementos de y no son actualizados atómicamente en este ejemplo.

```
int main()
{
float x[1000];
float y[10000];
int index[10000];
int i;
for (i = 0; i < 10000; i++) {
index[i] = i % 1000;
y[i]=0.0;
}
for (i = 0; i < 1000; i++)
x[i] = 0.0;
atomic_example(x, y, index,
10000);
return 0;
}
```

Constructor ordered

Especifica un bloque en un ciclo que será ejecutado en el orden de las iteraciones en el ciclo, mientras el programa se ejecuta en paralelo fuera de la región.

```
#pragma omp ordered new-line
structured-block
```

Ordered influencia todos los *threads* de la región, en el ciclo interior inmediato. Mientras regiones **ordered** en distintos ciclos se ejecutan independientemente. Cuando un *thread* ejecutando la primera iteración encuentra un constructor **ordered**, ingresa sin esperar. Cuando un *thread* ejecutando una iteración posterior encuentra una región **ordered**, espera a que las iteraciones anteriores han sido completadas.
El ciclo perteneciente a una región **ordered** debe poseer una cláusula **ordered** declarada en el constructor correspondiente.

Constructor y cláusula ordered

Constructores *ordered*
son útiles para ordenar
secuencialmente
la salida de
una región en paralelo.
El ejemplo
imprime los índices
en orden secuencial.

```
#include <stdio.h>
void work(int k)
{
#pragma omp ordered
printf(" %d\n", k);
}
void ordered_example(int lb, int ub, int stride)
{
int i;
#pragma omp parallel for
ordered schedule(dynamic)
for (i=lb; i<ub; i+=stride)
work(i);
}
int main() {
ordered_example(0, 100, 5);
return 0;
}
```

Constructor y cláusula ordered

Es posible tener múltiples constructores **ordered** en un ciclo con la cláusula **ordered** declarada.

Sin embargo, en este caso todas las iteraciones ejecutan dos regiones **ordered**, por lo que el código **no es válido**. Una iteración en un ciclo no debe ejecutar mas de una región **ordered**.

```
void work(int i) {}
void ordered_wrong(int n)
{
int i;
#pragma omp for ordered
for (i=0; i<n; i++) {
/* incorrect because an iteration may
not execute more than one ordered region */
#pragma omp ordered
work(i);
#pragma omp ordered
work(i+1);
}
}
```

Constructor y cláusula ordered

Este código (con
más de un constructor **ordered**)
si es válido, ya que cada iteración
ejecutará solo una región **ordered**

```
void work(int i) {}
void ordered_good(int n)
{
int i;
#pragma omp for ordered
for (i=0; i<n; i++) {
if (i <= 10) {
#pragma omp ordered
work(i);
}
if (i > 10) {
#pragma omp ordered
work(i+1);
}
}
}
```

Clausula private

Declara uno o más elementos de lista como privados para una tarea (*task*)

Cada *task* que referencia un elemento **private** recibe una lista copia del original con referencias a su nueva lista. El valor original podrá cambiar si se accesa por punteros.

Una variable que es parte de otra (como un *array* o *structure*) no puede ser parte de la lista de una cláusula **private**.

Clausula private

En el ejemplo,
los valores de la lista
de elementos originales *i*
y j se mantienen al salir de
la region paralela, mientras
que los elementos privados
iy j son modificados dentro
del constructor parallel.

```
#include <stdio.h>
#include <assert.h>
int main()
{
int i, j;
int *ptr_i, *ptr_j;
i = 1;
j = 2;
ptr_i = &i;
ptr_j = &j;
#pragma omp parallel private(i) firstprivate(j)
{
i = 3;
j = j + 2;
assert (*ptr_i == 1 && *ptr_j == 2);
}
assert(i == 1 && j == 2);
return 0;
}
```

Clausulas firstprivate y lastprivate

Cláusula **firstprivate**: Declara una lista privada a una tarea e inicializa cada elemento de la lista con el valor correspondiente de la variable original al momento que el constructor es inicializado. Se aplican restricciones de la cláusula **private**.
Cláusula **lastprivate**: Declara una lista privada a una tarea e inicializa cada elemento de la lista con el valor correspondiente de la variable original actualizada luego del final de la región. Se aplican restricciones de la cláusula **private**.

Cláusula lastprivate

Programas en los que la ejecución depende del valor en la última iteración, utilizan un listado de variables en una cláusula **lastprivate**, para que los valores sean equivalentes a los obtenidos en una ejecución secuencial.

```
void lastpriv (int n, float *a, float *b)
{
int i;
#pragma omp parallel
{
#pragma omp for lastprivate(i)
for (i=0; i<n-1; i++)
a[i] = b[i] + b[i+1];
}
a[i]=b[i]; /* i == n-1 here */
}
```

Directiva threadprivate

Especifica que variables serán replicadas, con cada *thread* teniendo su propia copia.

#pragma omp threadprivate(*list*) *new-line*

Cada copia de una variable **threadprivate** es inicializada una vez
Se usa para mantener una variable global en cada *thread* durante la ejecución de regiones en paralelo

Directiva threadprivate

```
#include <omp.h>
int  a, b, i, tid;
float x;
#pragma omp threadprivate(a, x);
main () {
/* Explicitly turn off dynamic threads */
  omp_set_dynamic(0);
  printf("1st Parallel Region:\n");
#pragma omp parallel private(b,tid) {
  tid = omp_get_thread_num();
  a = tid;
  b = tid;
  x = 1.1 * tid +1.0;
  printf("Thread %d:   a,b,x= %d %d %f\n",tid,a,b,x);
  } /* end of parallel section */
printf("Master thread doing serial work here\n");
printf("2nd Parallel Region:\n");
#pragma omp parallel private(tid)  {
  tid = omp_get_thread_num();
  printf("Thread %d:   a,b,x= %d %d %f\n",tid,a,b,x);
  } /* end of parallel section */ }
```

THREADPRIVATE:

Se usa para mantener una variable global en cada thread durante la ejecución de regiones en paralelo.

`#pragma omp threadprivate (list)`

```
1st Parallel Region:
Thread 0:   a,b,x= 0 0 1.000000
Thread 2:   a,b,x= 2 2 3.200000
Thread 3:   a,b,x= 3 3 4.300000
Thread 1:   a,b,x= 1 1 2.100000
Master thread doing serial work here
2nd Parallel Region:
Thread 0:   a,b,x= 0 0 1.000000
Thread 3:   a,b,x= 3 0 4.300000
Thread 1:   a,b,x= 1 0 2.100000
Thread 2:   a,b,x= 2 0 3.200000
```

Clausula reduction

Especifica un operador y una lista de elementos, para cada uno de los cuales una copia privada es creada en cada tarea (*task*) e inicializada para el operador. Al final de la región, el valor del elemento original es actualizado con los valores de las copias privadas utilizando el operador.

Operator	Initialization value
+	0
*	1
-	0
&	~0
|	0
^	0
&&	1
||	0
max	Least representable value in the reduction list item type
min	Largest representable value in the reduction list item type

Clausula reduction

El código no es correcto, ya que la inicialización (**a=0**) de la variable **a** original no esta sincronizada con la actualización de **a** como resultado de la cláusula **reduction** en el ciclo **for**. Por lo que el código imprimirá un valor errado de **a**. Para evitar el problema, la inicialización debe hacerse antes de la actualización debido a la claúsula **reduction**. Para ello se puede añadir una directiva **barrier** luego de **a=0**, o incluir **a=0** en una directiva **single** (que tiene barrera intrínseca), o inicializar a antes de iniciar la región **parallel**.

```
#include <stdio.h>
int main (void)
{
int a, i;
#pragma omp parallel shared(a) private(i)
{
#pragma omp master
a = 0;
// To avoid race conditions, add a barrier here.
#pragma omp for reduction(+:a)
for (i = 0; i < 10; i++) {
a += i;
}
#pragma omp single
printf ("Sum is %d\n", a);
}
}
```

Bibliografía i

- https://www.openmp.org/
- Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP.* 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.
- Barbara Chapman, Gabriele Jost, and Ruud van der Pas: *Using OpenMP*. MIT Press, 2008, ISBN-13: 978-0-262-53302-7.
- Georg Hager, Gerhard Wellein: *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press, 2010, 256p, ISBN13: 978-1-439-81192-4
- R.Chandra, L. Dagum, D. Kohr, D. Maydan, J. McDonald, and R. Menon: *Parallel programming in OpenMP*. Academic Press, San Diego, USA, 2000, ISBN 1-55860-671-8

UTEC
UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

